



[BIT703 Web Technologies](#) / [Contents](#) / Topic 10: Server-side web application development with procedural and object-oriented database programming / PHP OOP

Constructor and destructor functions 15m

Constructor function

Constructor functions are a special type of function, which is called automatically **whenever an object is created (often referred to as instantiation)**. In other words, they construct the object when it is instantiated. They are crucial for objects because they create and set the necessary attributes of the object when it is created (constructor) and free up any resources when it is destroyed (destructor).

To create a constructor function, PHP provides a special function called `__construct()` to define a constructor. You can pass as many arguments into this constructor function as you like. Notice that the construct function is prefixed with two underscores `__`.

Basically, when you instantiate an object of a class that uses the constructor function, the constructor function will be automatically called. If the constructor function has defined any arguments, these arguments will apply to the object at the time it is created. An advantage of the constructor function is that you can initialise many things.

If you wanted to create a constructor for the Book class we've been discussing, then you can use `__construct()` to initialise the price and title of the books at the time of object creation. To do this, you'd use the code below:

```
function __construct( $par1, $par2 ) {  
  
    $this->title = $par1;  
  
    $this->price = $par2;  
}
```

Skip to main content

```
}
```

Using this constructor function means that we don't need to use the `setPrice()`, `setTitle()` and similar functions separately to set the values of the attributes. Instead, we can initialise these attributes (the price, title and author variables) at the time of object creation through the constructor function.

The example below shows how using the constructor function reduces the amount of code used because there is no need to use the `setTitle`, `setPrice` or similar functions when the constructor function is used.

```
computerbk-> new Book( "Computer Basics for Dummies", 10, "Author A"
);

$mathsbk -> new Book( "Advanced Algebra", 15, "Author B");

$phpbk -> new Book( "PHP – A Complete Guide", 7, "Author C" );

/* Get those set values */

$computerbk ->getTitle();

$mathsbk ->getTitle();

$phpbk->getTitle();

$computerbk ->getPrice();

$mathsbk ->getPrice();

$phpbk ->getPrice();

$computerbk ->getAuthor();

$mathsbk -> getAuthor ();

$phpbk -> getAuthor ();
```

[Skip to main content](#)



☒ Constructor function

Non-Assessed

In this activity, execute the code for the Book class example using the constructor function.

Instead of using separate functions to set the title, price and author, you must execute the code for the Book class using the **constructor** function to initialise the variables of a book at the time of object creation.

Once you've executed the code using the constructor function, answer these questions:

- What is the output of this code?
- Does the output differ from the original code that used the setTitle, setPrice and setAuthor functions?
- ☐ I have successfully completed this task.

Submit



[Skip to main content](#)

Destructor function

The destructor function is the other special function, which is automatically called **at the end of a script (when you are done with the object)**. As mentioned earlier, this function lets the object free up any resources that may have been allocated BEFORE it is removed from memory.

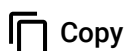
To create a destructor function, PHP uses a special function called `__destruct()`. Unlike the constructor function, a destructor doesn't accept any arguments. Notice that the destruct function is also prefixed with two underscores `__`.

A destructor is called when the object is destroyed/freed, or the script is stopped or exited. If you create a `__destruct()` function, PHP will automatically call this function at the end of the script. This function is **useful when code has made the last reference to an object or when a script reaches the end**.

Although the destructor function is not used often, it is important to know this feature exists, especially when your code/object is allocating resources, like with a database query, and you need to release the resources once you are done.

Using the `unset()` function, you can 'destroy' object instances without closing the running script. This might be useful when you have long-running scripts and want better control of the resources used while they are running.

The following example demonstrates the syntax for the destructor added to the Book class, though it is a trivial example.



Copy

[Skip to main content](#)

```
1  <?php
2      class Book {
3          var $price;
4          var $title;
5          var $author;
6
7          function __construct( $_title, $_price, $_author) {
8              $this->title = $_title;
9              $this->price = $_price;
10             $this->author = $_author;
11         }
12
13         function getTitle(){
14             echo $this->title . " <br/>";
15         }
16
17         function __destruct(){
18             /* free any resources allocate*/
19             echo "This book is called {$this->title}<br/>";
20         }
21
22         /* Methods */
23     }
24
25
26     $computerbk = new Book("Computer Basics for Dummies", 10);
27     $computerbk ->getTitle();
```



This code will produce the output:

Computer Basics for Dummies
Advanced Algebra
PHP – A Complete Guide
10
15
7

Skip to main content This book is called PHP – A Complete Guide and costs 7 dollars

This book is called Advanced Algebra and costs 15 dollars

This book is called Computer Basics for Dummies and costs 10 dollars

Can you see how the destructor function was automatically executed at the end of the script?

The following task will allow you to combine what you've learnt about classes, objects, calling member functions, constructor functions and destructor functions.



[Skip to main content](#)

☒ Classes, objects and functions

Non-Assessed

In this activity, you will create the code to meet the following criteria. Your code must:

- contain a class called Car, which consists of the attributes \$brand and \$colour
- create the objects \$Ford and \$Holden that are instances of the class Car that have the attributes shown below:
 - \$Ford – brand is Ford and colour is blue
 - \$Holden – brand is Holden and colour is red
- use the methods to get the \$brand and \$colour attributes
- use the constructor to automatically set the \$brand and \$colour attributes without needing to call on the setBrand or setColour methods.

☐ I have successfully completed the task

Submit



Let's summarise what we have discussed so far:

- A class is a template for objects, and an object is an instance of the class. A class contains the blueprint of how to build an object as well as all the information that defines the object.
- The object is the computer's **runtime** representation of the data describing the object and the methods to manipulate the object. There is a strong correlation between these two items, but it is important to remember that objects exist when the program is being run, and the class file is written by the programmer to allow the computer to use/construct new objects.

[Skip to main content](#)

Next, we'll look at inheritance.



Thoughts on this page?

